4.1 PPT

4.1 Context Engineering：把模型每一轮需要的信息组织好

- Loop 负责持续推进，Harness 管理运行，Runtime 执行工具。
- 任务能否完成，还取决于每轮给模型什么信息。
- 本节核心：装配、优先级、预算、调试报告。

1. Prompt Engineering 与 Context Engineering

登录会话修复任务

- 业务规则：`now < expires_at` 时有效，到期立即失效。
- 修复流程：定位问题、新增回归测试、修复实现、运行验证。
- 用户限制：不得修改现有测试，不得延长有效期绕过问题。
- 必要背景：代码入口、session 定义、测试命令、工作目录、代码版本。

每轮请求的组成

- Prompt：写清规则与要求。
- Context：提供当前代码、失败证据、任务进度、历史和工具说明。
- Prompt 是 Context 中的指令部分。

代码块

```
request = {
    "messages": [
        {"role": "system", "content": "项目规则与材料处理规则"},
        {"role": "user", "content": "当前用户目标和约束"},
        # 已发生的 assistant 回复、tool_calls 和对应 tool 结果
        # Harness 提供的当前任务状态
    ],
    "tools": tool_snapshot,  # 本轮可见工具及其参数 Schema
}
```

2. 上下文窗口限制

- 容量：输入、输出和工具调用参数都需要空间。
- 质量：无关材料、过期证据、冲突规则可能降低效果。
- 长窗口不保证判断准确，过度删减也可能丢失依据。
- 目标：在预算内保留当前判断需要的信息。

3. 上下文装配（Context Packing）：信息进入哪里

- 直接提供：业务规则、用户限制、当前源码、失败断言。
- 外部保存：完整日志，请求中保留读取路径。
- 按需补读：暂时不需要的材料，后续通过工具读取。

装配示例

代码块

```
*import json
from pathlib import Path

# 完整日志保存在文件中，不全部发送给模型。
log_path = Path("artifacts/session_tests.log")
log_path.parent.mkdir(parents=True, exist_ok=True)
log_path.write_text(
    "启动完成\n"
    "依赖检查完成\n"
    "test_before_expiry: PASSED\n"
    "test_at_expiry: FAILED\n"
    "输入：now=100, expires_at=100\n"
    "预期：False，实际：True\n"
    "test_after_expiry: PASSED\n",
    encoding="utf-8",
)

# 本轮直接提供：规则、限制、当前代码和失败证据。
context = {
    "业务规则": "当前时间等于或大于到期时间时，会话立即失效。",
    "修改限制": "不得修改现有测试，可以新增回归测试。",
    "当前代码": "return now <= expires_at",
    "失败证据": {
        "用例": "test_at_expiry",
        "输入": {"now": 100, "expires_at": 100},
        "预期": False,
```

```
        "实际": True,
    },
    "完整日志路径": str(log_path.resolve()),
}

# 提供读取工具的定义，供模型需要时申请补读。
read_file_tool = {
    "type": "function",
    "function": {
        "name": "read_file",
        "description": "读取文件，需要查看完整日志时使用",
        "parameters": {
            "type": "object",
            "properties": {
                "path": {"type": "string"},
            },
            "required": ["path"],
        },
    },
}

request = {
    "messages": [
        {
            "role": "system",
            "content": "根据提供的代码和证据分析问题，遵守修改限制。",
        },
        {
            "role": "user",
            "content": "检查登录会话为什么到期后仍然有效。",
        },
        {
            "role": "user",
            "content": json.dumps(context, ensure_ascii=False),
        },
    ],
    "tools": [read_file_tool],
}

**print(json.dumps(request, ensure_ascii=False, indent=2))*
```

代码重点

- `context`：本轮判断所需的信息。
- 日志文件：保存完整材料，请求携带关键证据和路径。

- `tools`：提供读取工具说明，执行由 Loop 和 Runtime 完成。
- 示例使用教学数据，不执行测试，也不调用模型。

4. 上下文优先级（Context Prioritization）：空间优先给谁用

- 先检查有效性、当前相关性和遗漏风险。
- 持续保留用户约束，优先提供当前源码和失败证据。
- 旧探索可压缩，无关材料可暂缓。
- 必须保留的是必要信息，完整材料可以外存。

材料	本轮处理	原因
用户目标、修改限制	必须保留	明确修复范围
当前失败、必要源码	保留关键内容	支持定位到期边界错误
当前任务状态	保留精简视图	说明补丁和验证进度
已完成的探索	压缩为结论、理由和来源	减少重复探索
旧版通过记录	不作为当前验证依据	不能证明当前代码正确
无关支付文档	本轮不提供	对当前判断没有帮助

分配顺序

代码块

```python
def allocation_order(blocks):
    return sorted(
        range(len(blocks)),
        key=lambda i: (
            not blocks[i].required,   # 必须项优先
            -blocks[i].priority,      # 数值越大，越先分配
            i,                        # 相同优先级保持原顺序
        ),
    )
```

材料	处理
用户目标和限制、业务规则、当前源码	必须保留
当前失败日志	保留失败断言和完整日志路径
当前任务状态	用剩余名额保留
旧版通过记录	排除，已失效
早期探索	优先级较低，本轮不提供

优先级与消息顺序

- 分配顺序决定先给谁空间，历史仍按原交互顺序组装。
- 工具调用与结果保持配对。
- 排查阶段重证据，修改后重 Diff 和新测试。
- 关键约束保留理由，执行权限由 Runtime 检查。

面试核心

最近的信息不一定最重要。按目标、证据有效性和遗漏风险选择材料，再检查预算。

5. 上下文预算（Context Budget）：本轮能放多少信息（Tokens）

预算公式

输入预算 = 上下文窗口容量 − 输出预留 − 安全余量

- 还须满足模型与 Provider 的独立输入、输出上限。
- 示例：16,000 − 3,000 − 1,000 = 12,000 Token。
- 固定规则、用户要求、工具定义和格式开销都计入输入。
- 固定内容占 3,000，其他材料可用 9,000。

预算与计数

代码块

```
def sample_request():
    *return* {
        "messages": [
```

```
            {"role": "system", "content": "根据当前代码和证据分析问题，遵守修改限制。"},
            {"role": "user", "content": json.dumps({
                "目标": "修复登录会话到期后仍有效的问题。",
                "规则": "now >= expires_at 时立即失效。",
                "代码": "return now <= expires_at",
                "失败": "now=100, expires_at=100; 预期 False，实际 True。",
                "限制": "不得修改现有测试，可新增回归测试。",
            }, ensure_ascii=False)},
        ],
        "tools": [{"type": "function", "function": {
            "name": "read_file", "description": "读取文件内容",
            "parameters": {"type": "object", "properties": {
                "path": {"type": "string"},
            }, "required": ["path"]},
        }}],
    }
```

装配运行结果

代码块

```
课堂容量 1000 - 输出 128 - 余量 150 = 输入预算 722
全部装入的本地估算： 17183
尝试 业务规则和限制/完整：105 / 722
尝试 当前失败/完整：12137 / 722
尝试 当前失败/压缩：183 / 722
尝试 当前源码/完整：199 / 722
排除 旧测试：证据已失效
尝试 当前状态/完整：220 / 722
尝试 支付文档/完整：5230 / 722
跳过 支付文档：剩余预算不足
装配后的本地估算： 220
```

结果解读

- 教学输入预算：1,000 − 128 − 150 = 722。
- 全部材料约 17,183 Token，发送前先调整。
- 长失败日志压缩，旧测试排除，支付文档跳过。
- 最终本地估算约 220～222，路径变化会影响计数。

项目	Token 数
实际输入 prompt_tokens	131
实际输出 completion_tokens	128
总消耗 total_tokens	259

服务端 usage 与超限拦截

代码块

```
尝试 必要证据/完整: 12116 / 722
尝试 必要证据/压缩: 6108 / 722
已拦截: 必须项 必要证据 压缩后仍超限，拒绝发送
DeepSeek 实际 usage:
{"prompt_tokens":131,"completion_tokens":128,"total_tokens":259,"prompt_tokens_
details":
{"cached_tokens":0},"prompt_cache_hit_tokens":0,"prompt_cache_miss_tokens":131}
实际输入与估算之差: -90
实际输入是否在课堂预算内: True
```

计数口径

- 本地估算 221，实际输入 131，差值不能固定套用。
- 输入预算对照 `prompt_tokens`，整次消耗看 `total_tokens`。
- 发送前估算，发送后用 usage 校准计数和安全余量。
- 每次尝试打印的是加入材料后整个请求的占用。

溢出处理

- 必须项优先，可以压缩，可选项放不下可跳过。
- 基础规则放不下，或必须项压缩后仍超限，停止发送。
- 缩小代码范围、减少重复规则、只提供本轮工具，必要时拆步骤。
- 换模型后重新检查窗口、能力与预算。
- Harness 同时累计整个任务的 Token、费用和时间。

6. 上下文压缩（Context Compression）：缩短内容，保留依据

两种压缩方式

- 工具结果：程序提取结构化关键字段。
- 任务历史：模型生成交接摘要，保留继续工作的线索。
- Pi 默认压缩较早历史、保留近期消息，并结合旧摘要更新。
- Codex 按配置与 Provider 能力采用模型摘要或远端压缩。

工具结果的必要字段

- 执行状态、失败用例、输入、预期、实际结果。
- 受测代码版本、异常信息、完整记录位置。
- 总失败数与省略数量，避免把部分结果当作全部。

提取短结果

代码块

```
def compact_test_result(result, max_failures=3):
    if max_failures < 1:
        raise ValueError("至少保留一条失败证据")

    # 字段缺失直接报错，不能当作测试通过。
    keys = (
        "revision",
        "status",
        "tests_run",
        "failure_count",
        "error_count",
        "errors",
        "output_ref",
    )
    summary = {key: result[key] for key in keys}

    failures = result["failures"]
    summary["failures"] = failures[:max_failures]
    summary["omitted_failures"] = (
        len(failures) - len(summary["failures"])
    )
    return summary
```

实验结果

- 三个真实边界检查，一个失败，保留恰好到期断言。
- 完整结果约 3.6 万字符，短结果约 459 字符。
- 重复启动信息用于模拟长日志，测试判断来自真实执行。
- 代码变化后重新测试，短结果仍须检查预算。

任务摘要示例

```
目标：修复会话到期后仍然有效的问题。
限制：不得修改现有测试。
已确认：当前实现使用 <=，恰好到期的测试失败。
已排除：延长有效期违反业务要求。
当前进度：补丁尚未应用。
下一步：修改边界判断并验证。
来源：相关源码、测试版本和完整结果路径。
```

压缩后的检查

- 示例展示应保留的内容，并非前面脚本生成的模型摘要。
- 关键约束由程序保留，测试状态以执行记录为准。
- 更新摘要时结合新增记录和可核对事实。
- 源码按需读范围，路径、版本和工具调用 ID 准确保留。
- 摘要调用计入 Token、费用和时间。

7. 上下文污染：避免错误信息影响判断

- 过期信息：旧测试不能证明修改后的代码正确。
- 信息冲突：标明来源，明确当前采用的规则。
- 摘要失真：区分猜测与已确认事实，保留原文核对入口。
- 外部指令：文件、网页内容不构成用户授权，操作仍需权限检查。

工具消息检查

- 调用与结果必须配对，结构完整后再发送。
- 消息结构正确，仍须检查证据版本和内容有效性。
- 课堂代码对不完整工具消息直接报错。

代码块

```
def check(messages):
    pending, seen = set(), set()
    *for* message *in* messages:
        *if* pending and message["role"] != "tool":
            *raise* ValueError(f"INCOMPLETE_TOOL_BATCH: {sorted(pending)}")
        *if* message["role"] == "tool":
            cid = message.get("tool_call_id")
            *if* cid not in pending:
                *raise* ValueError(f"ORPHAN_TOOL_RESULT: {cid}")
            pending.remove(cid)
        *for* call *in* message.get("tool_calls", []):
            cid = call.get("id")
            *if* message["role"] != "assistant" or not cid or cid in seen:
                *raise* ValueError(f"INVALID_TOOL_CALL_ID: {cid}")
            seen.add(cid)
            pending.add(cid)
    *if* pending:
        *raise* ValueError(f"MISSING_TOOL_RESULT: {sorted(pending)}")
```

中断后的处理

- 根据执行记录补齐中断状态，保留原调用 ID。
- 部分结果标记为不完整。
- 补齐消息修复历史，重新读取属于新的工具调用。

8. 上下文调试报告（Context Debug Report）：查清本轮给了模型什么

报告记录什么

- 材料来源、处理方式、选择原因。
- 最终请求、输入预算、最终估算、计数方法、请求哈希。
- 总输入从完整请求重新计算，不直接相加单项占用。
- 完整执行记录保留，进入模型的内容可以截短。

记录项	用途
名称和来源	找到原始材料
代码版本	判断证据是否仍然有效
保留、压缩或丢弃	知道程序做了什么
处理原因	知道为什么这样做
原始和保留内容的估算占用	观察压缩效果

演示：压缩历史后继续修复

代码块

```
python 4-1-8-Report.py
```

原始运行输出

代码块

```
(AgentLoop) wilson@WilsonM1 4-1 % python3 4-1-8-report.py
01-inspect usage: {"completion_tokens": 17, "prompt_cache_hit_tokens": 0, "prompt_cache_miss_tokens": 323, "prompt_tokens": 323, "prompt_tokens_details": {"cached_tokens": 0}, "total_tokens": 340}
02-diagnose usage: {"completion_tokens": 208, "prompt_cache_hit_tokens": 0, "prompt_cache_miss_tokens": 412, "prompt_tokens": 412, "prompt_tokens_details": {"cached_tokens": 0}, "total_tokens": 620}
问题已定位。

**问题位置**：`session.py` 中的 `is_active` 函数

```python
def is_active(now, expires_at):
    return now <= expires_at
```

问题原因：使用了 `<=`（小于等于）。当 `now == expires_at`（恰好到期）时，条件成立返回 `True`，即到期瞬间会话仍然有效。

正确逻辑：到期瞬间会话应失效，应使用严格小于 `<`：

```python
```
```

```
def is_active(now, expires_at):
    return now < expires_at
```

测试证据:
- `test_at`: `is_active(100, 100)` 期望 `False` → 当前返回 `True` (FAIL)
- `test_before`: 未到期应有效 (ok)
- `test_after`: 已过期应失效 (ok)

仅 `<=` 改为 `<` 即可修复。如需我应用补丁并重新运行测试，请告知。
03-compact usage: {"completion_tokens": 176, "prompt_cache_hit_tokens": 512, "prompt_cache_miss_tokens": 160, "prompt_tokens": 672, "prompt_tokens_details": {"cached_tokens": 512}, "total_tokens": 848}
目标：修复 session.py 中登录会话到期瞬间仍有效的问题。
限制：不得修改测试。
真实失败：test_at 失败，is_active(100,100) 返回 True，期望 False。
当前版本：2b2bcb44fec5aa5f60775a650e72b45a5e1cbed2d24a802710ff4a2986362c00
日志：/Users/wilson/Desktop/AI Agent 全栈工程师训练营/正式课内容/code/week04/4-1/codex_deepseek_runs/7b1f5220/test-009b6667.log
原因：is_active 用 <=，应为 <。
未做：未应用补丁，未重跑测试。
下一步：将 <= 改为 <，运行测试验证。
04-before-probe usage: {"completion_tokens": 32, "prompt_cache_hit_tokens": 384, "prompt_cache_miss_tokens": 253, "prompt_tokens": 637, "prompt_tokens_details": {"cached_tokens": 384}, "total_tokens": 669}
问题已定位在 `session.py` 的 `is_active` 使用了 `<=`（到期瞬间仍返回 True），尚未修复、未重新验证
05-after-probe usage: {"completion_tokens": 29, "prompt_cache_hit_tokens": 0, "prompt_cache_miss_tokens": 262, "prompt_tokens": 262, "prompt_tokens_details": {"cached_tokens": 0}, "total_tokens": 291}
session.py 中 is_active 的到期判断使用 `<=`，导致会话在恰好到期时仍返回 True；尚未应用修复。
06-resume-0 usage: {"completion_tokens": 42, "prompt_cache_hit_tokens": 256, "prompt_cache_miss_tokens": 421, "prompt_tokens": 677, "prompt_tokens_details": {"cached_tokens": 256}, "total_tokens": 719}
I'll start by inspecting the project to verify the current code and failure described in the handoff summary.
06-resume-1 usage: {"completion_tokens": 112, "prompt_cache_hit_tokens": 640, "prompt_cache_miss_tokens": 410, "prompt_tokens": 1050, "prompt_tokens_details": {"cached_tokens": 640}, "total_tokens": 1162}
Inspection confirms the handoff summary. The revision hash matches, `is_active` uses `<=`, and `is_active(100, 100)` returns `True` while `test_at` expects `False`. Now I'll apply the fix.
06-resume-2 usage: {"completion_tokens": 44, "prompt_cache_hit_tokens": 1024, "prompt_cache_miss_tokens": 199, "prompt_tokens": 1223, "prompt_tokens_details": {"cached_tokens": 1024}, "total_tokens": 1267}

```
The patch applied, producing new revision `9c45e87f...`. Now verifying with the real test run.
06-resume-3 usage: {"completion_tokens": 310, "prompt_cache_hit_tokens": 1152, "prompt_cache_miss_tokens": 320, "prompt_tokens": 1472, "prompt_tokens_details": {"cached_tokens": 1152}, "total_tokens": 1782}
修复完成，并已通过真实测试验证。

**定位问题（inspect_project 实测）**
- 修订版本 `2b2bcb44...` 与交接摘要一致
- 源码：`def is_active(now, expires_at): return now <= expires_at`
- 失败：`test_at` 中 `is_active(100, 100)` 返回 `True`，断言 `assertFalse` 失败
- 确认根因：`<=` 使到期瞬间（`now == expires_at`）仍被判为有效

**修复（apply_patch）**
- 将 `return now <= expires_at` 改为 `return now < expires_at`
- 新修订版本：`9c45e87f...`

**验证（run_test 实测）**
- 退出码：`0`
- 结果：`Ran 3 tests ... OK`
- `test_at`、`test_before`、`test_after` 全部通过
- 日志：`/Users/wilson/Desktop/AI Agent 全栈工程师训练营/正式课内容/code/week04/4-1/codex_deepseek_runs/7b1f5220/test-32a3cf5c.log`

边界语义现为：`now == expires_at` 时会话无效，满足"到期瞬间会话必须无效"。测试文件未改动，仅修改了 `session.py` 中的边界比较。
真实验收通过：模型摘要后继续修复，测试通过，测试文件未修改。
产物： /Users/wilson/Desktop/AI Agent 全栈工程师训练营/正式课内容/code/week04/4-1/codex_deepseek_runs/7b1f5220
```

第一步，确认真实失败。

第二步，生成交接摘要。

第三步，用摘要重建输入。

第四步，比较实际输入占用。

输入方式	实际输入 Token
完整历史	637
摘要重建后的历史	262

这个结果证明本轮输入变短了，不能直接证明整个任务更省钱。

第五步，继续修复并验证。

运行后看哪些文件：

文件	查看内容
original-history.json	压缩前的真实消息和工具结果
summary.md	DeepSeek 生成的交接摘要
rebuilt-history.json	保留规则、用户要求和摘要后的输入
*-request.json	每次实际发送的请求
*-response.json	模型回复、usage、请求哈希和耗时
context-report.json	压缩前后 Token、历史处理方式和最终验收结果

重点看三个结果：

摘要保留了真实状态。

重建后的输入确实变短了。

模型能接着工作，但仍然重新核对证据。

模型判断错了，怎样查

假设恢复后的模型说“测试已经通过”，就按顺序查：

1. `original-history.json` 中，真实测试结果是什么？
2. `summary.md` 有没有把“失败”写成“通过”，或者漏掉“尚未修复”？
3. 恢复后的 `*-request.json` 有没有带上正确摘要和用户限制？
4. 对应响应是否仍然作出了错误判断？

接入现有 Agent

- Harness 保存每轮请求，关联响应、usage 和耗时。
- 压缩时保存原历史、摘要、重建历史及压缩原因。
- Pi 负责 Loop，Runtime 执行工具。
- 正式工程关联任务 ID、轮次、策略版本、实际模型和工具快照。

9. 检索上下文（Retrieval Context）：缺什么，再去找

- 缺日志：根据路径用 `read_file` 读取失败附近。
- 缺定义：用 `search_code` 定位，再读取必要范围。
- 检查来源、版本、相关性、访问权限，再按优先级和预算装配。

- 现有仓库工具即可起步。
- 分块、Embedding、混合检索和重排留到 4.4。

10. 长上下文的成本与延迟

- 每轮发送完整历史，累计 Token 增长，响应也可能变慢。
- 压缩后输入变短，摘要调用仍有成本。
- 比较整个任务：完成情况、累计 Token、总费用、首文字延迟、总耗时。
- 模型等待与生成、工具执行分别计时。

11. KV 缓存（KV Cache）

- KV 是注意力计算中的 Key 和 Value。
- 保存已完成的计算，生成后续 Token 时复用。
- 学习目的：理解相似请求为何可能更快、输入布局怎样影响复用。
- 部分服务支持跨请求前缀缓存，命中条件以服务实现为准。
- 稳定规则与工具保持内容和顺序一致，动态状态放后面。
- 规则、权限、工具变化时及时更新，信息正确优先。

12. LiteLLM

调用职责

- Harness 整理输入，LiteLLM 按配置处理路由、限流、并发和重试。
- KV 与前缀缓存复用计算，响应缓存复用已有回答。
- 代码版本、权限、工具变化须进入请求或缓存隔离规则。

调用前检查

- 输入超限：回到装配器，减少材料或拆步骤。
- 切换模型：重检窗口与能力。
- 模型重试：统一管理，不能重放已执行的写操作。

原始运行输出

代码块

```
路由：agent-default → DeepSeek/deepseek-flash；thinking=disabled
演示输入是固定教学材料；所有模型响应均来自真实调用或此前真实响应的缓存。

--- 1-首次请求 ---
边界判断不正确。依据：规则要求“到期立即失效”，即 `now == expires_at` 时应为 False；但 `now <= expires_at` 在恰好到期时返回 True，与测试预期 False 冲突。
调用记录：{"label": "1-首次请求", "route": "agent-default", "target_model": "deepseek-flash", "cache_enabled": true, "cache_hit": false, "usage": {"completion_tokens": 64, "prompt_tokens": 116, "total_tokens": 180, "completion_tokens_details": {"accepted_prediction_tokens": null, "audio_tokens": null, "reasoning_tokens": 0, "rejected_prediction_tokens": null, "text_tokens": null, "image_tokens": null, "video_tokens": null}, "prompt_tokens_details": null}, "finish_reason": "stop", "status": "ok", "first_text_ms": 1234, "total_ms": 1514}

--- 2-相同输入检查缓存 ---
边界判断不正确。依据：规则要求“到期立即失效”，即 `now == expires_at` 时应为 False；但 `now <= expires_at` 在恰好到期时返回 True，与测试预期 False 冲突。
调用记录：{"label": "2-相同输入检查缓存", "route": "agent-default", "target_model": "deepseek-flash", "cache_enabled": true, "cache_hit": true, "usage": {"completion_tokens": 64, "prompt_tokens": 116, "total_tokens": 180, "completion_tokens_details": {"accepted_prediction_tokens": null, "audio_tokens": null, "reasoning_tokens": 0, "rejected_prediction_tokens": null, "text_tokens": null, "image_tokens": null, "video_tokens": null}, "prompt_tokens_details": null}, "finish_reason": "stop", "status": "ok", "first_text_ms": 5, "total_ms": 8}
缓存检查：  命中

--- 3-版本改变重新调用 ---
边界判断正确。

依据：条件是“到期立即失效”，即 `now == expires_at` 时应失效。`now < expires_at` 在相等时为 false，正好失效；三个边界测试均通过也支持该判断。
调用记录：{"label": "3-版本改变重新调用", "route": "agent-default", "target_model": "deepseek-flash", "cache_enabled": true, "cache_hit": false, "usage": {"completion_tokens": 68, "prompt_tokens": 110, "total_tokens": 178, "completion_tokens_details": {"accepted_prediction_tokens": null, "audio_tokens": null, "reasoning_tokens": 0, "rejected_prediction_tokens": null, "text_tokens": null, "image_tokens": null, "video_tokens": null}, "prompt_tokens_details": null}, "finish_reason": "stop", "status": "ok", "first_text_ms": 663, "total_ms": 1001}
版本变化检查：  未复用旧缓存
```

```
10:07:46 - LiteLLM:WARNING: utils.py:2898 - register_model: model=720e3b387f6b544c6e9e727f696fd57633975ab5fa50617f687adfe8375b3fb7 not in built-in cost map and no prefix/region variant matched; cache cost fields will default to 0. To track cache cost, add cache_creation_input_token_cost and cache_read_input_token_cost to model_info

--- 4-并发请求-1 ---

--- 4-并发请求-2 ---

调用记录: {"label": "4-并发请求-1", "route": "agent-default", "target_model": "deepseek-flash", "cache_enabled": false, "cache_hit": false, "usage": {"completion_tokens": 47, "prompt_tokens": 76, "total_tokens": 123, "completion_tokens_details": {"accepted_prediction_tokens": null, "audio_tokens": null, "reasoning_tokens": 0, "rejected_prediction_tokens": null, "text_tokens": null, "image_tokens": null, "video_tokens": null}, "prompt_tokens_details": null}, "finish_reason": "stop", "status": "ok", "first_text_ms": 539, "total_ms": 864}

调用记录: {"label": "4-并发请求-2", "route": "agent-default", "target_model": "deepseek-flash", "cache_enabled": false, "cache_hit": false, "usage": {"completion_tokens": 47, "prompt_tokens": 76, "total_tokens": 123, "completion_tokens_details": {"accepted_prediction_tokens": null, "audio_tokens": null, "reasoning_tokens": 0, "rejected_prediction_tokens": null, "text_tokens": null, "image_tokens": null, "video_tokens": null}, "prompt_tokens_details": null}, "finish_reason": "stop", "status": "ok", "first_text_ms": 722, "total_ms": 987}
并发实验结果:  ['ok', 'ok']
10:07:47 - LiteLLM:WARNING: utils.py:2898 - register_model: model=855b8a7463035ad8e0f9bf00ae9306a95c56df7274bd386eedc6105f8e45a236 not in built-in cost map and no prefix/region variant matched; cache cost fields will default to 0. To track cache cost, add cache_creation_input_token_cost and cache_read_input_token_cost to model_info

--- 5-限流请求-1 ---

调用记录: {"label": "5-限流请求-1", "route": "agent-default", "target_model": "deepseek-flash", "cache_enabled": false, "cache_hit": false, "usage": {"completion_tokens": 1, "prompt_tokens": 66, "total_tokens": 67, "completion_tokens_details": {"accepted_prediction_tokens": null, "audio_tokens": null, "reasoning_tokens": 0, "rejected_prediction_tokens": null, "text_tokens": null, "image_tokens": null, "video_tokens": null}, "prompt_tokens_details": null}, "finish_reason": "stop", "status": "ok", "first_text_ms": 589, "total_ms": 648}

--- 5-限流请求-2 ---

```

```
调用记录：{"label": "5-限流请求-2", "route": "agent-default", "target_model": "deepseek-flash", "cache_enabled": false, "cache_hit": null, "usage": null, "finish_reason": null, "status": "error", "error_type": "RateLimitError", "http_status": 429, "first_text_ms": null, "total_ms": 7}
限流实验以记录为准；若恰好跨越分钟边界，两次可能都通过。
报告： code/week04/4-1/report.json
```

本版配置

- 直接使用 LiteLLM 库与现有 `DEEPSEEK_API_KEY`。
- `agent-default` 路由到 `deepseek-flash`，关闭 thinking。
- 固定教学输入，回复来自真实调用或真实响应缓存。

实际验证结果

- 响应缓存命中：首次 1,514 毫秒，再次 8 毫秒。
- 输入变化后缓存未命中，重新调用模型。
- 限流第二次返回 `429 RateLimitError`。
- 两个并发请求都成功，尚不能证明并发限制生效。
- 本次未展示重试过程或模型回退。
- 本实验没有实际改代码、运行测试。

调用记录与警告

- 记录缓存命中、usage、首文字延迟、总耗时、错误状态。
- 缓存 usage 来自原响应，不能重复计为新增模型消耗。
- usage 缺失记 `null`，流中途失败由 Harness 决定后续动作。
- 价格表缺失时默认零费用不代表免费，须补实际价格。
- `acompletion` 未等待警告仍须修复。

小结

- 核心能力：选择材料、安排优先级、检查预算、压缩、调试。
- 验收：限制不丢，当前失败入请求，旧证据失效，工具配对，必须项超限拒发。

- 面试能讲：更多上下文为何可能降低效果、怎样分配预算、压缩遗漏怎样排查。
- AI Coding 实现装配器与 Debug Report，工程师明确规则并验收。
- 下一节：Working Memory，保存和更新当前任务事实。